

JAVA 17 OCP · YDS STUDY SERIES

Unit 07 · Technical Memory Notes · Beyond Classes

Teknik İngilizceyi bağlam içinde öğren · kısa tekrarlarla kalıcılaştır

Bu dosya [eksiksiz çift dilli Chapter 7 notundaki](#) kuralları active recall ve karşılaştırma kartlarıyla pekiştirir. Ana model: interface contract verir, enum sınırlı instance kümesi verir, sealed type subtype kümesini sınırlar, record state taşıyıcısını kısaltır. Dil çalışması: [vocabulary](#).

Kapsam denetimi

Ana çift dilli kaynak interface concrete methods, enum abstract methods, sealed hierarchy, record constructors, nested classes ve polymorphism konularını tam kapsar. Bu hafıza notunda özellikle şu bağlar birlikte tekrar edilir:

- interface member'ların implicit modifiers tablosu,
- enum constant body ve constructor davranışı,
- canonical/compact/overloaded record constructor farkı,
- dört nested class türünün scope/outer-instance farkı,
- reference type, object type, casting ve `instanceof` karar akışı.

1. Interface member matrisi

Fields

Interface field'ları her zaman `public static final`dır ve declaration sırasında initializer ister.

```
interface Limits {
    int MAX = 10; // public static final
}
```

Methods

Declaration	Implicit access/modifier	Body	Implementing class'a inherited mı?
<code>void run();</code>	<code>public abstract</code>	Hayır	Contract olarak evet
<code>default void run() {}</code>	<code>public</code>	Evet	Evet
<code>static void help() {}</code>	<code>public</code>	Evet	Hayır; interface adıyla çağrılır
<code>private void log() {}</code>	<code>private</code>	Evet	Hayır
<code>private static void log() {}</code>	<code>private static</code>	Evet	Hayır

Interface method `protected` olamaz. Private methods interface içi helper'dır; abstract olamaz çünkü implementing type tarafından görünmez.

2. Default method conflict çözümü

İki unrelated interface aynı signature'lı default method verirse concrete class override ederek conflict'i çözmelidir.

```
interface Swim { default void move() { System.out.print("swim"); } }
interface Walk { default void move() { System.out.print("walk"); } }

class Penguin implements Swim, Walk {
    public void move() {
        Swim.super.move();
    }
}
```

Class method'u interface default method'una üstün gelir. Interface inheritance chain'inde daha specific interface default'u da daha general olana üstün gelir.

Hafıza cümlesi: İki default çarpışırsa class hakem olur.

3. Enum: sınırlı instance kümesi

```
enum Season {
    WINTER("cold"), SUMMER("hot");

    private final String description;

    Season(String description) {
        this.description = description;
    }
}
```

- Enum constants implicit `public static final` instances'dır.
- Enum constructor implicit `private`dir; `public/protected` olamaz.
- Constant list'ten sonra fields/methods varsa semicolon gerekir.
- `values()` declaration order'da yeni array döndürür.
- `name()` exact identifier, `ordinal()` zero-based position döndürür.
- Enum switch case label'ında enum type adı yazılmaz: `case WINTER`.

Constant-specific class body, belirli constant için method override edebilir. Enum abstract method bildirirse her constant implementation sağlamalıdır.

4. Sealed classes and interfaces

```
sealed class Shape permits Circle, Polygon {}
final class Circle extends Shape {}
non-sealed class Polygon extends Shape {}
```

Sealed type yalnız permitted direct subtypes'a izin verir. Her direct subtype şu üç modifier'dan birini açıkça seçer:

- `final`: hierarchy burada kapanır,
- `sealed`: yeni sınırlı bir permits halkası açılır,
- `non-sealed`: bu daldan sonra normal açık inheritance devam eder.

`permits` listesi omitted ise direct subtypes aynı compilation unit içinde bildirilerek compiler tarafından inferred edilebilir. Named module içinde permitted subtypes aynı module'da; unnamed module kullanılıyorsa aynı package'ta olmalıdır.

Sealed interface'i permitted class implement edebilir, permitted interface extend edebilir.

Hafıza cümlesi: Sealed kapıda liste tutar; child kapıyı kapatır, mühürler ya da `non-sealed` ile açar.

5. Records: state declaration'dan generated API'ye

```
public record Point(int x, int y) {}
```

Compiler temel olarak şunları sağlar:

- her component için `private final` field,
- component ile aynı adlı public accessor: `x()`, `y()`,
- canonical constructor,
- component state'ine dayalı `equals()`, `hashCode()`, `toString()`.

Record implicit `final`dır ve `java.lang.Record`u extend eder. Başka class extend edemez; interface implement edebilir. Extra instance field bildiremez, static field bildirebilir.

Canonical, compact ve overloaded constructors

```
public record Range(int min, int max) {
    public Range() { // compact canonical
        if (min > max) throw new IllegalArgumentException();
    }

    public Range(int value) { // overloaded
        this(value, value); // first statement delegation
    }
}
```

- Canonical constructor bütün components'ı exact declaration order/type ile alır.
- Compact constructor parentheses yazmaz; implicit field assignments body'den sonra yapılır.
- Compact body'de parameter yeniden atanabilir; `this.component` field'ına explicit assignment yapılamaz.
- Non-canonical/overloaded constructor ilk statement'ta `this(...)` ile başka record constructor'a delegate etmelidir.
- Explicit accessor return type'ı component type ile exact eşleşmelidir.

Hafıza cümlesi: Record component'i field + accessor + constructor parameter'ını aynı satırda doğurur.

6. Dört nested class türü

Tür	Bildirildiği yer	Outer instance gerekir mi?	Özel not
Inner class	class body, non-static	Evet	<code>outer.new Inner()</code>
Static nested class	class body, static	Hayır	<code>new Outer.Nested()</code>
Local class	method/block	Context'e bağlı	effectively final locals capture eder
Anonymous class	expression	Context'e bağlı	adı/explicit constructor'ı yok

Java 16'dan itibaren inner class static members bildirebilir; eski “yalnız constant static member” ezberi Java 17 için yanlıştır.

Anonymous class bir class'ı extend eder veya bir interface'i implement eder. `final` class'tan anonymous subclass oluşturulamaz. Explicit constructor bildiremez; initializer block kullanabilir.

7. Polymorphism: reference kapaıyı, object davranışı belirler

```
Animal animal = new Dog();
```

- Compile-time access: `Animal` type'ında görünür member'lar.
- Overridden instance method execution: runtime `Dog` object type'ı.
- Hidden field/static method selection: compile-time reference/context.
- Overload selection: compile-time argument/reference types.

İşlem	Compile-time sonucu	Runtime riski
Upcast <code>Dog → Animal</code>	implicit geçerli	Yok
Downcast <code>Animal → Dog</code>	explicit cast gerekir	Object Dog değilse <code>ClassCastException</code>
Unrelated class cast	çoğunlukla compilation error	Çalışmaya ulaşmaz
<code>null instanceof Type</code>	geçerli	<code>false</code>

```
if (animal instanceof Dog dog) {
    dog.fetch();
}
```

Pattern variable yalnız `match`'in kesin olduğu flow scope'ta kullanılabilir.

8. Encapsulation bağlantısı

Encapsulation yalnız fields'ı `private` yapmak değildir; object'in valid state kurallarını controlled methods/constructors üzerinden korumaktır. Record shallow immutability sağlar: component reference finaldır, fakat gösterdiği mutable object otomatik immutable olmaz. Gerekirse defensive copy uygulanır.

OCP için 20 saniyelik analiz sırası

- 1 Interface member'ın implicit modifiers'ını açıkça yaz.
- 2 Default conflict veya missing abstract implementation var mı?
- 3 Enum constant list semicolon ve constructor access doğru mu?
- 4 Sealed direct subtype permitted mı ve `final/sealed/non-sealed` seçmiş mi?
- 5 Record constructor canonical mı; değilse `this(...)` ile delegate ediyor mu?
- 6 Nested class için outer instance gerekiyor mu?
- 7 Member compile-time reference'a mı, runtime object'e mi bağlı?
- 8 Downcast compile olsa bile actual object compatible mı?

Aktif hatırlama · Özgün çalışma soruları

- 1 Interface static method implementing class tarafından inherited olur mu?
- 2 Permitted sealed subclass modifier yazmadan bırakılabilir mi?
- 3 Record compact constructor field assignment'ı ne zaman gerçekleşir?
- 4 `Animal a = new Cat(); Dog d = (Dog) a;` compile ve runtime sonucu nedir?

Cevaplar

- 1 Hayır; interface adı üzerinden çağrılır.
- 2 Hayır; `final`, `sealed` veya `non-sealed` seçmelidir.
- 3 Compact constructor body tamamlandıktan sonra implicit olarak.
- 4 Class'lar ilişkiliyse cast compile olabilir; actual object `Cat` olduğu için runtime'da `ClassCastException` oluşur.